NETWORKS LAB PRACTICAL

Common Python Templates + Topic-Specific Logic

Exam revision codebook • TCP, UDP, chat, multi-chat, ARP, DNS, ARQ, CSMA, routing and related topics

How to use this PDF: Learn the common TCP and UDP structures first. For each question, keep the network setup and replace the marked processing logic. Some topics (GBN, SR, CSMA, subnetting and routing) are simulations or standalone algorithms and cannot be solved by changing only a message string.

Requirements: Python 3. For SimPy examples: `pip install simpy`. Run server first, then client, in separate terminals.

1. Common TCP Server

```
import socket

HOST = "127.0.0.1"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(5)
print("TCP server started")

while True:
    conn, addr = server.accept()
    with conn:
        print("Connected:", addr)
        data = conn.recv(1024).decode()
        print("Client:", data)

        # TOPIC LOGIC GOES HERE
        reply = "Request received"

        conn.sendall(reply.encode())
```

Use for TCP request-response programs. Change HOST/PORT if needed. Replace only the reply logic for simple lookup tasks; use a loop for chat and threading for multi-client chat.

2. Common TCP Client

```
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

message = input("Enter message: ")
client.sendall(message.encode())

reply = client.recv(1024).decode()
print("Server:", reply)
client.close()
```

3. Common UDP Server

```
import socket

HOST = "127.0.0.1"
PORT = 5001

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))
print("UDP server started")

while True:
    data, addr = server.recvfrom(1024)
    message = data.decode()
    print("Client:", message)

    # TOPIC LOGIC GOES HERE
    reply = "Request received"

    server.sendto(reply.encode(), addr)
```

4. Common UDP Client

```
import socket

HOST = "127.0.0.1"
PORT = 5001

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = input("Enter message: ")
client.sendto(message.encode(), (HOST, PORT))
data, addr = client.recvfrom(1024)
print("Server:", data.decode())
client.close()
```

5. Common SimPy Structure

```
import simpy

def process(env, name, resource):
    start = env.now
    print(name, "requests at", env.now)
    with resource.request() as req:
        yield req
        print(name, "starts at", env.now)
        # Replace with protocol-specific steps
        yield env.timeout(1)
        print(name, "finishes at", env.now)
    print("Time taken:", env.now - start)

env = simpy.Environment()
resource = simpy.Resource(env, capacity=1)
for i in range(3):
    env.process(process(env, "Node " + str(i + 1), resource))
env.run()
```

SimPy is useful for timing/resource simulations, but a generic resource template alone does not implement GBN, SR or CSMA. Those need protocol-specific state and events.

6. Topic Logic — Quick Reference

Topic	Logic to add / change
TCP request-response	Read request; validate/process it; send a reply.
UDP request-response	Use recvfrom() and sendto(); no accept()/connect() handshake.
TCP chat	Repeat send/receive until /quit; both peers must follow the same turn order.
UDP chat	Repeat sendto()/recvfrom(); handle timeouts because UDP does not guarantee delivery.
Multi-client chat	Use threading.Thread per accepted connection; maintain client list; broadcast messages.
ARP	Use MAC-to-IP table; normalize MAC case; return matching IP or not found.
DNS	Use domain-to-IP table; normalize domain to lowercase; return IP or not found.
Stop-and-Wait ARQ	Send one frame; wait for ACK; on timeout retry up to retry limit.
Go-Back-N	Maintain base and next sequence; if frame lost, retransmit all outstanding frames from lost frame.
Selective Repeat	Track ACK per frame; buffer valid frames; retransmit only unacknowledged/lost frames.
Checksum	Calculate checksum on sender; receiver recomputes and compares. Additive checksum is not CRC.
CRC	Implement XOR/polynomial division; append remainder; receiver checks remainder.
Bit stuffing	Insert 0 after five consecutive 1s; destuff by removing inserted 0.
CSMA	Sense channel; if idle transmit; if multiple stations transmit together, count collision.
CSMA/CA	Use random backoff, contention window, ACK/retry logic; this differs from simple CSMA.
Packet routing	Check destination against routes; select longest-prefix match; return next hop/interface.
Subnetting	Parse IPv4 CIDR; calculate network, broadcast, prefix and usable host range.
Flow control	Use receiver window/buffer; sender must not send more unacknowledged data than allowed.
Congestion control	Track cwnd and threshold; slow start grows exponentially, congestion avoidance additively.
Network performance	Compute loss=sent-received; loss rate; throughput=received/time; average delay.
Wireless IoT	Generate sensor readings; package ID/value; simulate sending and threshold alerts.

7. TCP Chat — Alternating messages

This is a simple alternating chat: client sends first, server replies second. Run server.py, then client.py. Type /quit to stop.

Server

```
import socket
HOST, PORT = "127.0.0.1", 6203

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(1)
    print("Waiting for client...")
    conn, addr = s.accept()
    with conn:
        print("Connected:", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            msg = data.decode()
            print("Client:", msg)
            if msg == "/quit":
                break
            reply = input("You: ")
            conn.sendall(reply.encode())
            if reply == "/quit":
                break
```

Client

```
import socket
HOST, PORT = "127.0.0.1", 6203

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    while True:
        msg = input("You: ")
        s.sendall(msg.encode())
        if msg == "/quit":
            break
        data = s.recv(1024)
        if not data:
            break
        reply = data.decode()
        print("Server:", reply)
        if reply == "/quit":
            break
```

8. ARP and DNS Lookup Logic

ARP: MAC address to IP address

```
arp_table = {
    "00:11:22:33:44:55": "192.168.1.10",
    "AA:BB:CC:DD:EE:FF": "192.168.1.20"
}
mac = input("Enter MAC: ").strip().upper()
print("IP address:", arp_table.get(mac, "MAC not found"))
```

DNS: domain to IP address

```
dns_table = {
    "example.com": "93.184.216.34",
    "college.edu": "192.0.2.10"
}
domain = input("Enter domain: ").strip().lower()
print("IP address:", dns_table.get(domain, "Domain not found"))
```

These are educational lookup tables. They do not query the real operating-system ARP cache or perform full recursive DNS resolution.

9. Stop-and-Wait ARQ

```
frames = ["F1", "F2", "F3"]
lost_ack_frame = 2
max_retries = 3
```

```
for number, frame in enumerate(frames, start=1):
    for attempt in range(1, max_retries + 1):
        print("Send", frame, "attempt", attempt)
        if number == lost_ack_frame and attempt == 1:
            print("ACK lost; timeout")
            continue
        print("ACK received for", frame)
        break
    else:
        print("Frame dropped:", frame)
```

Change the frame list, retry limit and simulated lost ACK/frame to demonstrate the scenario.

10. Go-Back-N (GBN)

```
total_frames = 8
window_size = 4
lost_frame = 3
loss_triggered = False
base = 1

while base <= total_frames:
    end = min(base + window_size - 1, total_frames)
    print("Send window:", base, "to", end)
    failed = False
    for frame in range(base, end + 1):
        print("Send frame", frame)
        if frame == lost_frame and not loss_triggered:
            print("Frame lost:", frame)
            loss_triggered = True
            print("Timeout: retransmit from frame", frame)
            base = frame
            lost_frame = -1
            failed = True
            break
    if not failed:
        print("Cumulative ACK", end)
        base = end + 1
P("Teaching simulation of one loss. Full GBN needs timers/ACK state to represent real event timing.")
```

11. Selective Repeat (SR)

```
total_frames = 6
lost_frame = 3
received = set()
loss_triggered = False

for frame in range(1, total_frames + 1):
    print("Send frame", frame)
    if frame == lost_frame and not loss_triggered:
        print("Frame lost; no ACK")
        loss_triggered = True
    else:
        received.add(frame)
        print("ACK", frame)

for frame in range(1, total_frames + 1):
    if frame not in received:
        print("Retransmit only lost frame", frame)
        received.add(frame)
        print("ACK", frame)
print("All frames acknowledged:", len(received) == total_frames)
```

Teaching simulation of one loss. SR uses individual acknowledgements and receiver buffering; a full implementation needs per-frame timers and sequence-number window rules.

12. Bit Stuffing and Destuffing

```
def stuff(bits):
    result, ones = "", 0
    for bit in bits:
        result += bit
        if bit == "1":
            ones += 1
            if ones == 5:
                result += "0"
                ones = 0
        else:
            ones = 0
    return result
```

```
def destuff(bits):
    result, ones, i = "", 0, 0
    while i < len(bits):
        bit = bits[i]
        result += bit
        if bit == "1":
            ones += 1
            if ones == 5:
                if i + 1 < len(bits) and bits[i + 1] == "0":
                    i += 1
                ones = 0
        else:
            ones = 0
        i += 1
    return result

bits = input("Enter bit string: ").strip()
encoded = stuff(bits)
print("Stuffed:", encoded)
print("Destuffed:", destuff(encoded))
```

13. CSMA and CSMA/CA Logic

Simplified slotted CSMA

```
import random
random.seed(2)
stations = int(input("Number of stations: "))
slots = int(input("Number of slots: "))
p = float(input("Attempt probability (0-1): "))
success = collisions = idle = 0

for slot in range(1, slots + 1):
    active = [i for i in range(1, stations + 1) if random.random() < p]
    if len(active) == 0:
        idle += 1
        print(slot, "IDLE")
    elif len(active) == 1:
        success += 1
        print(slot, "SUCCESS station", active[0])
    else:
        collisions += 1
        print(slot, "COLLISION", active)

print("Success:", success, "Collisions:", collisions, "Idle:", idle)
```

Simplified CSMA/CA backoff

```
import random
random.seed(3)
cw_min, cw_max, max_retries = 4, 16, 4

for packet in range(1, 6):
    cw = cw_min
    for attempt in range(max_retries + 1):
        backoff = random.randint(0, cw - 1)
        print("Packet", packet, "attempt", attempt + 1, "backoff", backoff)
        collision = random.random() < 0.30
        if not collision:
            print("Success")
            break
        print("Collision")
        cw = min(cw * 2, cw_max)
    else:
        print("Packet dropped")
```

These are simplified educational models, not complete IEEE 802.11 simulations with carrier sensing, frozen counters, ACK timing and all MAC rules.

14. Packet Routing

```
import ipaddress

routes = [
    ("192.168.1.0/24", "192.168.1.1", "eth0"),
    ("10.0.0.0/8", "10.0.0.1", "eth1"),
    ("0.0.0.0/0", "192.168.1.1", "eth0")
]
dest = ipaddress.ip_address(input("Destination IPv4: "))
matches = []
for net_text, next_hop, interface in routes:
```

```
    net = ipaddress.ip_network(net_text)
    if dest in net:
        matches.append((net.prefixlen, net_text, next_hop, interface))

if matches:
    _, net, hop, iface = max(matches)
    print("Matched network:", net)
    print("Next hop:", hop)
    print("Interface:", iface)
else:
    print("No route; drop packet")
```

15. Subnetting

```
import ipaddress
cidr = input("Enter CIDR (e.g. 192.168.1.0/24): ")
net = ipaddress.ip_network(cidr, strict=False)
print("Network:", net.network_address)
print("Broadcast:", net.broadcast_address)
print("Prefix:", net.prefixlen)
print("Total addresses:", net.num_addresses)
if net.num_addresses > 2:
    print("First usable:", next(net.hosts()))
    print("Last usable:", list(net.hosts())[-1])
```

16. Checksum vs CRC

Additive checksum example

```
def checksum(message):
    return sum(message.encode("utf-8")) % 256

data = input("Enter message: ")
sent_checksum = checksum(data)
print("Checksum:", sent_checksum)
received_checksum = int(input("Enter checksum to verify: "))
print("Valid" if received_checksum == checksum(data) else "Error detected")
```

This is an additive checksum example. A true CRC question requires binary polynomial/XOR division and remainder checking; do not label this code as CRC.

17. Flow Control

```
frames = list(range(1, 11))
window = int(input("Receiver window size: "))
if window < 1:
    raise ValueError("Window must be >= 1")
for start in range(0, len(frames), window):
    batch = frames[start:start + window]
    print("Transmit:", batch)
    print("ACK through frame", batch[-1])
```

18. Congestion Control

```
threshold = int(input("Slow-start threshold (e.g. 8): "))
cwnd = 1
for round_no in range(1, 9):
    print("Round", round_no, "cwnd =", cwnd)
    if cwnd < threshold:
        cwnd *= 2
    else:
        cwnd += 1
```

This illustrates slow start and additive increase only. Full TCP congestion control also reacts to loss/ECN and uses algorithm-specific rules.

19. Network Performance Metrics

```
sent = int(input("Packets sent: "))
received = int(input("Packets received: "))
seconds = float(input("Duration in seconds: "))
avg_delay = float(input("Average delay in seconds: "))
lost = sent - received
print("Lost:", lost)
print("Packet loss %:", round(lost / sent * 100, 2) if sent else 0)
print("Throughput packets/s:", round(received / seconds, 3) if seconds > 0 else 0)
print("Average delay ms:", round(avg_delay * 1000, 3))
```

```
print("Delivery ratio %:", round(received / sent * 100, 2) if sent else 0)
```

20. Wireless IoT Sensor Logic

```
import random
random.seed(5)
count = int(input("Number of sensor readings: "))
threshold = 30

for sensor_id in range(1, count + 1):
    temperature = round(random.uniform(20, 40), 2)
    packet = {"sensor_id": sensor_id, "temperature": temperature}
    print("Transmit:", packet)
    if temperature > threshold:
        print("ALERT: high temperature")
    else:
        print("Normal")
```

21. Multi-client TCP Chat — Core Server Logic

```
import socket
import threading

HOST, PORT = "127.0.0.1", 6205
clients = []
lock = threading.Lock()

def broadcast(message, sender):
    with lock:
        targets = list(clients)
    for client in targets:
        if client is not sender:
            try:
                client.sendall(message)
            except OSError:
                pass

def handle_client(conn, addr):
    try:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            msg = data.decode().strip()
            if msg == "/quit":
                break
            broadcast(f"{addr[1]}: {msg}".encode(), conn)
    finally:
        with lock:
            if conn in clients:
                clients.remove(conn)
        conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen()
print("Multi-chat server started")
while True:
    conn, addr = server.accept()
    with lock:
        clients.append(conn)
    threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()
```

22. Exam Checklist

- Start the server before starting the client.
- TCP: SOCK_STREAM, bind, listen, accept, connect, sendall, recv.
- UDP: SOCK_DGRAM, sendto and recvfrom; no accept/listen.
- Chat: loop until /quit; multi-chat needs threads and broadcast.
- GBN: cumulative ACK; after loss, retransmit from the lost frame.
- SR: individual ACK; retransmit only missing/unacknowledged frame.
- CSMA: channel sensing and contention; CSMA/CA adds random backoff and retry logic.

- Routing: longest-prefix match selects the most specific matching route.
- Test code with small inputs and be ready to explain every major line.